Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа №2

на тему:

«Решение непрерывной выпукло-вогнутой антагонистической игры аналитическим и численным методами»

Вариант 1

Студент: ИУ8-105 (Группа) В.В.Гоза (И. О. Фамилия)

Преподаватель: Н. С. Коннова (И.О. Фамилия)

Цель работы — найти оптимальные стратегии непрерывной выпукло-вогнутой антагонистической игры аналитическим и численным методами.

Постановка задачи:

Требуется исследовать антагонистическую игру, заданную на единичном квадрате

$$\Pi = [0,1] \times [0,1]$$

с ядром, заданным в условии задания:

Вариант задания: 1

Вариант	a	b	c	d	e
1	-5	5/12	10/3	-2/3	-4/3

Ход работы:

1) **Аналитическое решение**

Рассматриваем ядро игры в виде

$$H(x,y) = -5x^2 + \frac{5}{12}y^2 + \frac{10}{3}xy - \frac{2}{3}x - \frac{4}{3}y, \qquad x, y \in [0,1]$$

Проверка условий выпукло-вогнутой игры

Найдём вторые производные:

$$H_{xx} = 2a = -10 < 0, \;\; H_{yy} = 2b = \frac{5}{6} > 0$$

Следовательно, функция выигрыша является вогнутой по переменной x и выпуклой по переменной y. Значит, игра является выпукло-вогнутой и допускает решение в чистых стратегиях.

Нахождение стационарной точки

Найдём первые производные:

$$H_x = -10x + \frac{10}{3}y - \frac{2}{3}$$

$$H_y = \frac{5}{6}y + \frac{10}{3}x - \frac{4}{3}$$

В седловой точке внутри области $[0,1] \times [0,1]$ должно выполняться:

$$H_x = 0 \text{ и } H_y = 0$$

Решаем уравнение и получаем:

$$x^* = \frac{1}{5} = 0{,}2$$

$$y^* = \frac{4}{5} = 0{,}8$$

Подставим найденную точку в ядро:

$$H\left(\frac{1}{5}, \frac{4}{5}\right) = -\frac{3}{5} = -0{,}6$$

Следовательно, аналитическое решение:

$$x^* = 0{,}2 \quad y^* = 0{,}8 \quad H^* = -\frac{3}{5} = -0{,}6$$

```
============================================================
АНАЛИТИЧЕСКОЕ РЕШЕНИЕ
============================================================

Частные производные функции H(x,y):
  ∂H/∂x = -10.0000x + 3.3333y - 0.6667
  ∂H/∂y = 3.3333x + 0.8333y - 1.3333

Стационарная точка: x* = 0.2000, y* = 0.8000
Значение функции: H* = -0.6000
```

Рисунок 1. Вывод программы аналитического решения

2) Численное решение

Для численного решения непрерывная игра заменяется матричной игрой на сетке:

$$x_i = N_i, \qquad y_i = N_j, \qquad i, j = 0, 1, \dots, N.$$

Элементы матрицы аппроксимирующей игры имеют вид:

$$a_{ij} = H\left(\frac{i}{N}, \frac{j}{N}\right)$$

Для каждой матрицы выполнялись следующие действия:

1. строилась матрица $A^{(N)}$;
2. проверялось наличие седловой точки;
3. если седловая точка отсутствовала, решение искалось методом Брауна–Робинсона;
4. затем вычислялась цена игры H;
5. процесс продолжался до тех пор, пока суммарное изменение цены игры за последние k=4 шагов по N не становилось меньше ε=0.01:

$$\sum_{r=t-3}^{t} |H_r - H_{r-1}| \leq 0.041$$

Вывод программы (седловая точка выделена зелёным цветом):

```
============================================================
N = 2 (шаг сетки = 1/2)
============================================================

Матрица игры:
            0        1        2
   0   0.0000  -0.5625  -0.9167
   1  -1.5833  -1.3125  -0.8333
   2  -5.6667  -4.5625  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 10
Нижняя оценка: -0.9167
Верхняя оценка: -0.8250
Координаты по смешанным стратегиям: x=0.0000 y=0.9000
Средняя оценка цены игры: H=-0.8708

============================================================
N = 3 (шаг сетки = 1/3)
============================================================

Матрица игры:
            0        1        2        3
   0   0.0000  -0.3981  -0.7037  -0.9167
   1  -0.7778  -0.8056  -0.7407  -0.5833
   2  -2.6667  -2.3241  -1.8889  -1.3611
   3  -5.6667  -4.9537  -4.1481  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 9
Нижняя оценка: -0.7315
Верхняя оценка: -0.6319
Координаты по смешанным стратегиям: x=0.1852 y=0.8889
Средняя оценка цены игры: H=-0.6817
```

```
============================================================
N = 4 (шаг сетки = 1/4)
============================================================

Матрица игры:
            0        1        2        3        4
   0   0.0000  -0.3073  -0.5625  -0.7656  -0.9167
   1  -0.4792  -0.5781  -0.6250  -0.6198  -0.5625
   2  -1.5833  -1.4740  -1.3125  -1.0990  -0.8333
   3  -3.3125  -2.9948  -2.6250  -2.2031  -1.7292
   4  -5.6667  -5.1406  -4.5625  -3.9323  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 12
Нижняя оценка: -0.6563
Верхняя оценка: -0.5603
Координаты по смешанным стратегиям: x=0.1875 y=0.8958
Средняя оценка цены игры: H=-0.6083

============================================================
N = 5 (шаг сетки = 1/5)
============================================================

Матрица игры (выделена седловая точка):
            0        1        2        3        4        5
   0   0.0000  -0.2500  -0.4667  -0.6500  -0.8000  -0.9167
   1  -0.3333  -0.4500  -0.5333  -0.5833  -0.6000  -0.5833
   2  -1.0667  -1.0500  -1.0000  -0.9167  -0.8000  -0.6500
   3  -2.2000  -2.0500  -1.8667  -1.6500  -1.4000  -1.1167
   4  -3.7333  -3.4500  -3.1333  -2.7833  -2.4000  -1.9833
   5  -5.6667  -5.2500  -4.8000  -4.3167  -3.8000  -3.2500

Есть седловая точка:
x=0.2000 y=0.8000 H=-0.6000
```

```
============================================================
N = 6 (шаг сетки = 1/6)
============================================================

Матрица игры:
            0        1        2        3        4        5        6
   0   0.0000  -0.2106  -0.3981  -0.5625  -0.7037  -0.8218  -0.9167
   1  -0.2500  -0.3681  -0.4630  -0.5347  -0.5833  -0.6088  -0.6111
   2  -0.7778  -0.8032  -0.8056  -0.7847  -0.7407  -0.6736  -0.5833
   3  -1.5833  -1.5162  -1.4259  -1.3125  -1.1759  -1.0162  -0.8333
   4  -2.6667  -2.5069  -2.3241  -2.1181  -1.8889  -1.6366  -1.3611
   5  -4.0278  -3.7755  -3.5000  -3.2014  -2.8796  -2.5347  -2.1667
   6  -5.6667  -5.3218  -4.9537  -4.5625  -4.1481  -3.7106  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 10
Нижняя оценка: -0.6722
Верхняя оценка: -0.5750
Координаты по смешанным стратегиям: x=0.1333 y=0.9000
Средняя оценка цены игры: H=-0.6236

============================================================
N = 7 (шаг сетки = 1/7)
============================================================

Матрица игры:
            0        1        2        3        4        5        6        7
   0   0.0000  -0.1820  -0.3469  -0.4949  -0.6259  -0.7398  -0.8367  -0.9167
   1  -0.1973  -0.3112  -0.4082  -0.4881  -0.5510  -0.5969  -0.6259  -0.6378
   2  -0.5986  -0.6446  -0.6735  -0.6854  -0.6803  -0.6582  -0.6190  -0.5629
   3  -1.2041  -1.1820  -1.1429  -1.0867  -1.0136  -0.9235  -0.8163  -0.6922
   4  -2.0136  -1.9235  -1.8163  -1.6922  -1.5510  -1.3929  -1.2177  -1.0255
   5  -3.0272  -2.8690  -2.6939  -2.5017  -2.2925  -2.0663  -1.8231  -1.5629
   6  -4.2449  -4.0187  -3.7755  -3.5153  -3.2381  -2.9439  -2.6327  -2.3044
   7  -5.6667  -5.3724  -5.0612  -4.7330  -4.3878  -4.0255  -3.6463  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 11
Нижняя оценка: -0.6617
Верхняя оценка: -0.5680
Координаты по смешанным стратегиям: x=0.1688 y=0.9091
Средняя оценка цены игры: H=-0.6149
```

```
============================================================
N = 8 (шаг сетки = 1/8)
============================================================

Матрица игры:
            0        1        2        3        4        5        6        7        8
   0   0.0000  -0.1602  -0.3073  -0.4414  -0.5625  -0.6706  -0.7656  -0.8477  -0.9167
   1  -0.1615  -0.2695  -0.3646  -0.4466  -0.5156  -0.5716  -0.6146  -0.6445  -0.6615
   2  -0.4792  -0.5352  -0.5781  -0.6081  -0.6250  -0.6289  -0.6198  -0.5977  -0.5625
   3  -0.9531  -0.9570  -0.9479  -0.9258  -0.8906  -0.8424  -0.7812  -0.7070  -0.6198
   4  -1.5833  -1.5352  -1.4740  -1.3997  -1.3125  -1.2122  -1.0990  -0.9727  -0.8333
   5  -2.3698  -2.2695  -2.1562  -2.0299  -1.8906  -1.7383  -1.5729  -1.3945  -1.2031
   6  -3.3125  -3.1602  -2.9948  -2.8164  -2.6250  -2.4206  -2.2031  -1.9727  -1.7292
   7  -4.4115  -4.2070  -3.9896  -3.7591  -3.5156  -3.2591  -2.9896  -2.7070  -2.4115
   8  -5.6667  -5.4102  -5.1406  -4.8581  -4.5625  -4.2539  -3.9323  -3.5977  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 12
Нижняя оценка: -0.6510
Верхняя оценка: -0.5585
Координаты по смешанным стратегиям: x=0.1771 y=0.9062
Средняя оценка цены игры: H=-0.6048

============================================================
N = 9 (шаг сетки = 1/9)
============================================================

Матрица игры:
            0        1        2        3        4        5        6        7        8        9
   0   0.0000  -0.1430  -0.2757  -0.3981  -0.5103  -0.6121  -0.7037  -0.7850  -0.8560  -0.9167
   1  -0.1358  -0.2377  -0.3292  -0.4105  -0.4815  -0.5422  -0.5926  -0.6327  -0.6626  -0.6821
   2  -0.3951  -0.4558  -0.5062  -0.5463  -0.5761  -0.5957  -0.6049  -0.6039  -0.5926  -0.5710
   3  -0.7778  -0.7973  -0.8066  -0.8056  -0.7942  -0.7726  -0.7407  -0.6986  -0.6461  -0.5833
   4  -1.2840  -1.2623  -1.2305  -1.1883  -1.1358  -1.0730  -1.0000  -0.9167  -0.8230  -0.7191
   5  -1.9136  -1.8508  -1.7778  -1.6944  -1.6008  -1.4969  -1.3827  -1.2582  -1.1235  -0.9784
   6  -2.6667  -2.5628  -2.4486  -2.3241  -2.1893  -2.0442  -1.8889  -1.7233  -1.5473  -1.3611
   7  -3.5432  -3.3981  -3.2428  -3.0772  -2.9012  -2.7150  -2.5185  -2.3117  -2.0947  -1.8673
   8  -4.5432  -4.3570  -4.1605  -3.9537  -3.7366  -3.5093  -3.2716  -3.0237  -2.7654  -2.4969
   9  -5.6667  -5.4393  -5.2016  -4.9537  -4.6955  -4.4270  -4.1481  -3.8591  -3.5597  -3.2500

Седловой точки нет, решение методом Брауна-Робинсона:

Итог метода Брауна-Робинсон:
Число итераций: 11
Нижняя оценка: -0.6540
Верхняя оценка: -0.5550
Координаты по смешанным стратегиям: x=0.1616 y=0.9091
Средняя оценка цены игры: H=-0.6045
```

```
============================================================
N = 10 (шаг сетки = 1/10)
============================================================

Матрица игры (выделена седловая точка):
            0        1        2        3        4        5        6        7        8        9       10
   0   0.0000  -0.1292  -0.2500  -0.3625  -0.4667  -0.5625  -0.6500  -0.7292  -0.8000  -0.8625  -0.9167
   1  -0.1167  -0.2125  -0.3000  -0.3792  -0.4500  -0.5125  -0.5667  -0.6125  -0.6500  -0.6792  -0.7000
   2  -0.3333  -0.3958  -0.4500  -0.4958  -0.5333  -0.5625  -0.5833  -0.5958  -0.6000  -0.5958  -0.5833
   3  -0.6500  -0.6792  -0.7000  -0.7125  -0.7167  -0.7125  -0.7000  -0.6792  -0.6500  -0.6125  -0.5667
   4  -1.0667  -1.0625  -1.0500  -1.0292  -1.0000  -0.9625  -0.9167  -0.8625  -0.8000  -0.7292  -0.6500
   5  -1.5833  -1.5458  -1.5000  -1.4458  -1.3833  -1.3125  -1.2333  -1.1458  -1.0500  -0.9458  -0.8333
   6  -2.2000  -2.1292  -2.0500  -1.9625  -1.8667  -1.7625  -1.6500  -1.5292  -1.4000  -1.2625  -1.1167
   7  -2.9167  -2.8125  -2.7000  -2.5792  -2.4500  -2.3125  -2.1667  -2.0125  -1.8500  -1.6792  -1.5000
   8  -3.7333  -3.5958  -3.4500  -3.2958  -3.1333  -2.9625  -2.7833  -2.5958  -2.4000  -2.1958  -1.9833
   9  -4.6500  -4.4792  -4.3000  -4.1125  -3.9167  -3.7125  -3.5000  -3.2792  -3.0500  -2.8125  -2.5667
  10  -5.6667  -5.4625  -5.2500  -5.0292  -4.8000  -4.5625  -4.3167  -4.0625  -3.8000  -3.5292  -3.2500

Есть седловая точка:
x=0.2000 y=0.8000 H=-0.6000
```

Рис. 2-6 Вывод программы численного решения

Согласно выбранному критерию остановки, процесс был завершён на матрице с

$$N = 14$$

Численно при остановке процесса:

$$x_{57} \approx 0.1607 \qquad y_{57} \approx 0.9167 \quad H_{57} = -0.6051$$

Абсолютные погрешности:

$$|x_{14} - x^*| \approx 0.0393$$

$$|y_{14} - y^*| \approx 0.1167$$

$$|H_{14} - H^*| \approx 0.0051$$

Следовательно, численный метод даёт результат, очень близкий к аналитическому.

Вывод

В ходе лабораторной работы была исследована непрерывная выпукло-вогнутая антагонистическая игра с ядром

$H(x,y) = -5x^2 + \frac{5}{12}y^2 + \frac{10}{3}xy - \frac{2}{3}x - \frac{4}{3}y$

Аналитическим методом найдено точное решение:

$$x^* = 0{,}2 \quad y^* = 0{,}8 \quad H^* = -\frac{3}{5} = -0{,}6$$

Численное решение было получено путём аппроксимации ядра игры на сетке и последующего решения соответствующих матричных игр. Критерием остановки служило условие стабилизации цены игры за последние 4 шагов по параметру разбиения сетки N. Процесс завершился при N=14.

Полученное численное решение:

$$x_{14} \approx 0.0393 \quad y_{14} \approx 0.1167 \quad H_{14} = -0.0051$$

практически совпадает с аналитическим результатом. Это подтверждает корректность как аналитического, так и численного подходов.

ПРИЛОЖЕНИЕ А

Листинг (также приведён по ссылке https://github.com/NeAlien/Game_theory_10_sem):

```
import numpy as np


def fmt4(value):
    return f"{float(value):.4f}"


def vector_decimal_str(values):
    return "(" + ", ".join(fmt4(v) for v in values) + ")"


def first_argmax(values):
    best_i = 0
    best_v = values[0]
    for i in range(1, len(values)):
        if values[i] > best_v:
            best_v = values[i]
            best_i = i
    return best_i


def first_argmin(values):
    best_i = 0
    best_v = values[0]
    for i in range(1, len(values)):
        if values[i] < best_v:
            best_v = values[i]
            best_i = i
    return best_i


a = -5.0
b = 5.0 / 12.0
```

```
c = 10.0 / 3.0
d = -2.0 / 3.0
e = -4.0 / 3.0

EPS = 0.01
K_STEPS = 4
MAX_N = 50

BR_EPS = 0.1
BR_MAX_STEPS = 10000

USE_COLOR = True
RESET = "\033[0m"
SADDLE_COLOR = "\033[30;102m"
TITLE_COLOR = "\033[1;36m"
INFO_COLOR = "\033[1;33m"


def colorize(text, color):
    if USE_COLOR:
        return f"{color}{text}{RESET}"
    return text


def H(x, y):
    return a * x * x + b * y * y + c * x * y + d * x + e * y


class ContinuousGameSolver:
    def __init__(self, a_coef, b_coef, c_coef, d_coef, e_coef):
        self.a = a_coef
        self.b = b_coef
        self.c = c_coef
        self.d = d_coef
        self.e = e_coef
```

```
        self.gridstep = None

    def setka_maker(self, N):
        x = np.linspace(0.0, 1.0, N + 1)
        y = np.linspace(0.0, 1.0, N + 1)
        self.gridstep = x

        X, Y = np.meshgrid(x, y)
        grid = np.column_stack([X.ravel(), Y.ravel()])
        return grid

    def solve(self, N):
        grid = self.setka_maker(N)
        x_vals = grid[:, 0]
        y_vals = grid[:, 1]

        z_vals = (
            self.a * x_vals ** 2
            + self.b * y_vals ** 2
            + self.c * x_vals * y_vals
            + self.d * x_vals
            + self.e * y_vals
        )

        size = len(self.gridstep)
        return z_vals.reshape(size, size).T


def find_saddle_points(matrix, tol=1e-10):
    matrix = np.array(matrix, dtype=float)

    row_mins = matrix.min(axis=1)
    col_maxs = matrix.max(axis=0)

    alpha = row_mins.max()
```

```
    beta = col_maxs.min()

    if not np.isclose(alpha, beta, atol=tol):
        return [], alpha, beta, row_mins, col_maxs

    saddle_points = []
    rows, cols = matrix.shape

    for i in range(rows):
        for j in range(cols):
            value = matrix[i, j]
            if np.isclose(value, row_mins[i], atol=tol) and np.isclose(value, col_maxs[j], atol=tol):
                saddle_points.append((i, j, value))

    return saddle_points, alpha, beta, row_mins, col_maxs


def analytical_solution():
    matrix_A = np.array([
        [2 * a, c],
        [c, 2 * b]
    ], dtype=float)

    vector_B = np.array([-d, -e], dtype=float)

    try:
        x_star, y_star = np.linalg.solve(matrix_A, vector_B)
        h_star = H(x_star, y_star)
        return x_star, y_star, h_star
    except np.linalg.LinAlgError:
        return None, None, None


def brown_robinson(matrix, eps=0.1, start_a=0, start_b=0, max_steps=10000):
    m, n = matrix.shape
```

```
count_A = [0] * m
count_B = [0] * n

win_A = [0] * m
loss_B = [0] * n

a_choice = start_a
b_choice = start_b

best_upper = None
best_lower = None
current_eps = None

table_rows = []

for k in range(1, max_steps + 1):
    count_A[a_choice] += 1
    count_B[b_choice] += 1

    for i in range(m):
        win_A[i] += matrix[i, b_choice]

    for j in range(n):
        loss_B[j] += matrix[a_choice, j]

    current_upper = max(win_A) / k
    current_lower = min(loss_B) / k

    if best_upper is None or current_upper < best_upper:
        best_upper = current_upper

    if best_lower is None or current_lower > best_lower:
        best_lower = current_lower
```

```
        current_eps = best_upper - best_lower

        table_rows.append({
            "k": k,
            "A_choice": f"x{a_choice + 1}",
            "B_choice": f"y{b_choice + 1}",
            "win_A": win_A.copy(),
            "loss_B": loss_B.copy(),
            "upper": current_upper,
            "lower": current_lower,
            "eps": current_eps,
            "count_A": count_A.copy(),
            "count_B": count_B.copy(),
        })

        if current_eps <= eps:
            break

        a_choice = first_argmax(win_A)
        b_choice = first_argmin(loss_B)

    return {
        "rows": table_rows,
        "iterations": len(table_rows),
        "count_A": count_A,
        "count_B": count_B,
        "best_upper": best_upper,
        "best_lower": best_lower,
        "eps": current_eps,
    }


def print_pretty_matrix(matrix, saddle_points=None):
    matrix = np.array(matrix, dtype=float)
    rows, cols = matrix.shape
```

```
    saddle_set = set()
    if saddle_points:
        for i, j, _ in saddle_points:
            saddle_set.add((i, j))

    row_index_width = max(4, len(str(rows - 1)) + 1)
    max_val_len = max(len(f"{matrix[i, j]:.4f}") for i in range(rows) for j in range(cols))
    cell_width = max(9, max_val_len + 2)

    title = "Матрица игры"
    if saddle_points:
        title += " (выделена седловая точка)"
    title += ":"

    print()
    print(colorize(title, TITLE_COLOR))

    header = " " * row_index_width
    for j in range(cols):
        header += f"{j:>{cell_width}}"
    print(header)

    for i in range(rows):
        line = f"{i:>{row_index_width}}"
        for j in range(cols):
            value_text = f"{matrix[i, j]:>{cell_width}.4f}"
            if (i, j) in saddle_set:
                value_text = colorize(value_text, SADDLE_COLOR)
            line += value_text
        print(line)


def print_derivative_formulas():
    print("\nЧастные производные функции H(x,y):")
```

```
    print(f"  ∂H/∂x = {fmt4(2 * a)}x + {fmt4(c)}y {('- ' + fmt4(abs(d))) if d < 0 else ('+ ' +
fmt4(d))}")
    print(f"  ∂H/∂y = {fmt4(c)}x + {fmt4(2 * b)}y {('- ' + fmt4(abs(e))) if e < 0 else ('+ ' +
fmt4(e))}")
    print()


def print_brown_robinson_summary(result, grid):
    k = result["iterations"]

    x_br = np.array([cnt / k for cnt in result["count_A"]], dtype=float)
    y_br = np.array([cnt / k for cnt in result["count_B"]], dtype=float)

    x_mean = float(np.dot(x_br, grid))
    y_mean = float(np.dot(y_br, grid))

    lower = float(result["best_lower"])
    upper = float(result["best_upper"])
    middle = (lower + upper) / 2.0

    print("\nИтог метода Брауна—Робинсон:")
    print(f"Число итераций: {k}")
    print(f"Нижняя оценка: {fmt4(lower)}")
    print(f"Верхняя оценка: {fmt4(upper)}")
    print(f"Координаты по смешанным стратегиям: x={fmt4(x_mean)} y={fmt4(y_mean)}")
    print(f"Средняя оценка цены игры: H={fmt4(middle)}")

    return x_mean, y_mean, middle


def main():
    solver = ContinuousGameSolver(a, b, c, d, e)

    print("\n" + "=" * 60)
    print(colorize("АНАЛИТИЧЕСКОЕ РЕШЕНИЕ", TITLE_COLOR))
```

```
print("=" * 60)

print_derivative_formulas()

x_analyt, y_analyt, h_analyt = analytical_solution()

if x_analyt is not None:
    print(f"Стационарная точка: x* = {x_analyt:.4f}, y* = {y_analyt:.4f}")
    print(f"Значение функции: H* = {h_analyt:.4f}")
else:
    print("Аналитическое решение недоступно.")
print()

print("=" * 60)
print(colorize("РЕШЕНИЕ ИГРЫ ДЛЯ N ОТ 2 ДО 50", TITLE_COLOR))
print("=" * 60)

H_history = []
last_numeric_x = None
last_numeric_y = None
last_numeric_h = None
last_N = None

for N in range(2, MAX_N + 1):
    print(f"\n{'=' * 60}")
    print(colorize(f"N = {N} (шаг сетки = 1/{N})", INFO_COLOR))
    print(f"{'=' * 60}")

    matrix = solver.solve(N)
    saddle_points, alpha, beta, row_mins, col_maxs = find_saddle_points(matrix)

    print_pretty_matrix(matrix, saddle_points)

    if saddle_points:
        row_idx, col_idx, _ = saddle_points[0]
```

```
            x_current = solver.gridstep[row_idx]
            y_current = solver.gridstep[col_idx]
            H_current = matrix[row_idx, col_idx]

            print("\nЕсть седловая точка:")
            print(f"x={x_current:.4f} y={y_current:.4f} H={H_current:.4f}")
        else:
            print("\nСедловой точки нет, решение методом Брауна-Робинсона:")

            result = brown_robinson(
                matrix=matrix,
                eps=BR_EPS,
                start_a=0,
                start_b=0,
                max_steps=BR_MAX_STEPS
            )

            x_current, y_current, H_current = print_brown_robinson_summary(result,
solver.gridstep)

        H_history.append(float(H_current))

        last_numeric_x = x_current
        last_numeric_y = y_current
        last_numeric_h = H_current
        last_N = N

        if len(H_history) >= K_STEPS:
            recent_H = H_history[-K_STEPS:]
            h_spread = max(recent_H) - min(recent_H)

            if h_spread <= EPS:
                print()
                print(f"Разброс H за последние {K_STEPS} шагов: {h_spread:.4f} <= {EPS:.4f}")
                print("Условие завершения выполнено.")
```

```
            break

    print("\n" + "=" * 60)
    print(colorize("СРАВНЕНИЕ АНАЛИТИЧЕСКОГО И ЧИСЛЕННОГО РЕШЕНИЙ",
TITLE_COLOR))
    print("=" * 60)

    if x_analyt is not None:
        print(f"Аналитическое решение: x={x_analyt:.4f} y={y_analyt:.4f} H={h_analyt:.4f}")

    if last_numeric_x is not None:
        print(f"Численное решение:    x={last_numeric_x:.4f} y={last_numeric_y:.4f}
H={last_numeric_h:.4f}")
        print(f"Последнее N = {last_N}")

    if x_analyt is not None and last_numeric_x is not None:
        print(f"|Δx| = {abs(x_analyt - last_numeric_x):.4f}")
        print(f"|Δy| = {abs(y_analyt - last_numeric_y):.4f}")
        print(f"|ΔH| = {abs(h_analyt - last_numeric_h):.4f}")


if __name__ == "__main__":
    main()
```